MLPs in PyTorch

DL-1 companion

Xingche Guo

STAT 5095 University of Connecticut · Fall 2026

Learning resources

Resource (click to open)	Use it for
Bourke: PyTorch Deep Learning learnpytorch.io (online book)	Guided notebooks, exercises, and a complete PyTorch workflow.
BIOS740 course syllabus `bios740.github.io/syllabus/`	Related deep learning lectures, starting with PyTorch and neural network fundamentals.
PyTorch: Learn the Basics	Official tutorials: datasets, models, autograd, optimization, and saving weights.
Dive into Deep Learning `d2l.ai`	An interactive textbook connecting the mathematics to PyTorch implementations.

What is PyTorch?

PyTorch is an **open-source framework for machine learning in Python**.

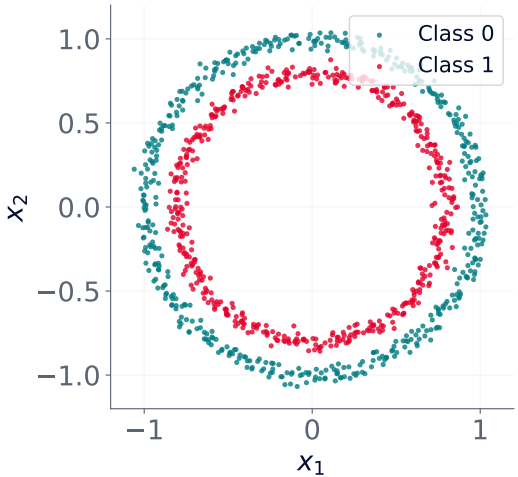
It combines CPU/GPU tensor computation with automatic differentiation.

Building block	**Role in our MLP examples**
`torch.Tensor`	Stores data and parameters as multidimensional arrays.
`torch.utils.data`	`Dataset` retrieves examples. `DataLoader` forms batches.
`torch.nn`	Layers and losses. `nn.Module` organizes a model and its parameters.
`torch.autograd`	Computes parameter gradients via `loss.backward()`.
`torch.optim`	Updates parameters, for example with SGD or Adam.

`torchvision` supplies FashionMNIST and the image transform `ToTensor()`.

Adapted from BIOS740 C1, pp. 46–49. See also PyTorch: Learn the Basics and Learning PyTorch with Examples.

Circles: why an MLP?



Input: two coordinates (x_1, x_2).

Target: inner or outer ring.

A single affine score gives a straight decision boundary.

An MLP learns nonlinear features that can separate the rings.

Prepare the circle data

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split

X, y = make_circles(1000, noise=0.03, random_state=42)
X, y = torch.from_numpy(X).float(), torch.from_numpy(y).float()
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

X_train, y_train = X_train.to(device), y_train.to(device)
X_test, y_test = X_test.to(device), y_test.to(device)
```

- Inputs are (x_1, x_2) coordinates. Targets are 0 or 1.
- The split keeps 800 observations for training and 200 for testing.
- `stratify=y` preserves the proportions of the classes.
- Move the split tensors to `device` before training.

One MLP layer in PyTorch

```
layer = nn.Linear(in_features=2, out_features=10).to(device)
hidden = torch.relu(layer(X_train))  # (800, 2) -> (800, 10)

layer.weight.shape  # (10, 2): 10 neurons, 2 weights each
layer.bias.shape    # (10,): one bias per neuron
```

$$h_{ij} = \mathrm{ReLU}(w_{j1}x_{i1} + w_{j2}x_{i2} + b_j), \qquad j = 1, \ldots, 10.$$

- `2` and `10` specify feature dimensions, independent of batch size.
- All observations share the same $10 \times 2 + 10 = 30$ trainable parameters.
- `Linear` supplies the affine transformation. ReLU adds nonlinearity.

Adapted from BIOS740 C2, pp. 23–26. Official API: torch.nn.Linear.

nn.Linear

`nn.Linear(in_features, out_features, bias=True)`

Shape rule: transform the last dimension; preserve the others.

$$(N, d_{\text{in}}) \longrightarrow (N, d_{\text{out}}).$$

Default initialization: each weight and bias is sampled from

$$W_{jk},\ b_j \sim \mathcal{U}\left(-\frac{1}{\sqrt{d_{\text{in}}}}, \frac{1}{\sqrt{d_{\text{in}}}}\right).$$

`layer.weight` and `layer.bias` are trainable parameters, automatically registered with the model.

The circle MLP

```
class TwoclassClassificationMLP(nn.Module):
    def __init__(self):
        super().__init__()  # Register layers and parameters
        self.hidden1 = nn.Linear(2, 10)
        self.hidden2 = nn.Linear(10, 10)
        self.output = nn.Linear(10, 1)
    def forward(self, x):
        x = torch.relu(self.hidden1(x))  # (B, 10)
        x = torch.relu(self.hidden2(x))  # (B, 10)
        return self.output(x)            # (B, 1)
```

- `__init__` creates the layers; `forward` defines their computation.
- The output is one logit. The binary loss handles the output sigmoid.
- ReLU between layers prevents collapse to one affine map.

Circle MLP with Sequential

```
torch.manual_seed(42)  # Set before constructing the layers
model_1 = nn.Sequential(
    nn.Linear(2, 10),
    nn.ReLU(),
    nn.Linear(10, 10),
    nn.ReLU(),
    nn.Linear(10, 1)   # Raw logit; no sigmoid here
).to(device)
```

- Same architecture: $2 \to 10 \to 10 \to 1$, with 151 trainable parameters.
- `Sequential` passes each layer's output directly to the next layer.
- Choose either definition, then use the same loss and training loop.

A custom `forward` is useful for branching, skip connections, or multiple outputs.

Subjects in columns or rows

DL-1 uses observations in columns:

$$H^{(l)} = \phi_l\Big(W^{(l)}H^{(l-1)} + b^{(l)}\mathbf{1}_B^\top\Big), \qquad H^{(l-1)} \in \mathbb{R}^{d_{\text{in}} \times B}.$$

Our PyTorch code uses observations in rows:

$$H_{\text{next}} = \phi\Big(HW^\top + \mathbf{1}_B b^\top\Big), \qquad H \in \mathbb{R}^{B \times d_{\text{in}}}.$$

Object	PyTorch shape	Meaning
Input	(B, d_{in})	B observations
Stored weight	$(d_{\text{out}}, d_{\text{in}})$	One row per output unit
Output	(B, d_{out})	One representation per observation

Binary logits and loss

```
logits = model_1(X_train).squeeze(-1)  # (B, 1) -> (B,)
loss_fn = nn.BCEWithLogitsLoss()
loss = loss_fn(logits, y_train)        # y_train: float, (B,)

probability = torch.sigmoid(logits)    # P(y = 1 | x)
label = (probability >= 0.5).long()    # Predicted class
```

$$p_i = \sigma(a_i), \qquad \ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i).$$

- A logit is a real-valued score. `BCELoss` takes sigmoid probabilities.
- `BCEWithLogitsLoss` takes raw logits and combines both steps stably.
- Threshold only for prediction. Keep raw logits connected to the training loss.

Activation choices

Activation	PyTorch (click)	Activation	PyTorch (click)
ReLU	`nn.ReLU()`	Sigmoid	`nn.Sigmoid()`
Leaky ReLU	`nn.LeakyReLU()`	Tanh	`nn.Tanh()`
PReLU	`nn.PReLU()`	Softplus	`nn.Softplus()`
ELU	`nn.ELU()`	GELU	`nn.GELU()`
SELU	`nn.SELU()`	SiLU / Swish	`nn.SiLU()`
ReLU6	`nn.ReLU6()`	Mish	`nn.Mish()`
Softmax	`nn.Softmax(dim=1)`	LogSoftmax	`nn.LogSoftmax(dim=1)`

Loss choices

Task / criterion	PyTorch loss (click)	Inputs to the loss
Squared error	`nn.MSELoss()`	Matching prediction and target shapes
Absolute error	`nn.L1Loss()`	Matching prediction and target shapes
Binary classes	`nn.BCEWithLogitsLoss()`	Raw logits and floating 0/1 labels
One of K classes	`nn.CrossEntropyLoss()`	K logits and integer class indices
KL divergence	`nn.KLDivLoss()`	Log-probabilities $\log Q$ and target probabilities P, with matching shapes

Optimizer choices

```
# Alternative constructors: select ONE for a training run.
optimizer = torch.optim.SGD(model_1.parameters(), lr=0.01)
optimizer = torch.optim.SGD(model_1.parameters(), lr=0.01,
                            momentum=0.9)
optimizer = torch.optim.Adagrad(model_1.parameters(), lr=0.01)
optimizer = torch.optim.RMSprop(model_1.parameters(), lr=0.001)
optimizer = torch.optim.Adam(model_1.parameters(), lr=0.001)
optimizer = torch.optim.AdamW(model_1.parameters(), lr=0.001,
                              weight_decay=0.01)
```

- `model_1.parameters()` identifies which tensors the optimizer updates.
- The optimizer changes the update rule. The same `backward()` computes gradients.
- These learning rates illustrate syntax. Compare suitable values using validation data.

Optimizers in PyTorch

Optimizer (click)	PyTorch code
SGD	`torch.optim.SGD(model_1.parameters(), lr=0.1)`
Momentum SGD	`torch.optim.SGD(model_1.parameters(), lr=0.01, momentum=0.9)`
Adagrad	`torch.optim.Adagrad(model_1.parameters(), lr=0.01)`
RMSprop	`torch.optim.RMSprop(model_1.parameters(), lr=0.001)`
Adam	`torch.optim.Adam(model_1.parameters(), lr=0.001)`
AdamW	`torch.optim.AdamW(model_1.parameters(), lr=0.001, weight_decay=0.01)`

Training the circle MLP

```
torch.manual_seed(42)
model_1 = TwoclassClassificationMLP().to(device)
loss_fn = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(
    model_1.parameters(), lr=0.1)

for epoch in range(2500):
    model_1.train()                    # 1
    logits = model_1(X_train).squeeze(-1)
    loss = loss_fn(logits, y_train)    # 2
    optimizer.zero_grad()              # 3
    loss.backward()                    # 4
    optimizer.step()                   # 5
```

One epoch = all 800 points = one update. The same loop works with `Sequential`.

1. **Training mode.** Enable training behavior, including dropout if present.
2. **Forward and loss.** Compute logits and mean binary cross-entropy.
3. **Clear gradients.** PyTorch adds gradients. Reset them before this backward pass.
4. **Backpropagate.** Compute the loss gradients for all weights and biases; save them in `.grad` without updating parameters.
5. **Update weights.** SGD applies $\theta \leftarrow \theta - 0.1 \nabla_\theta L$.

Evaluate the circle MLP

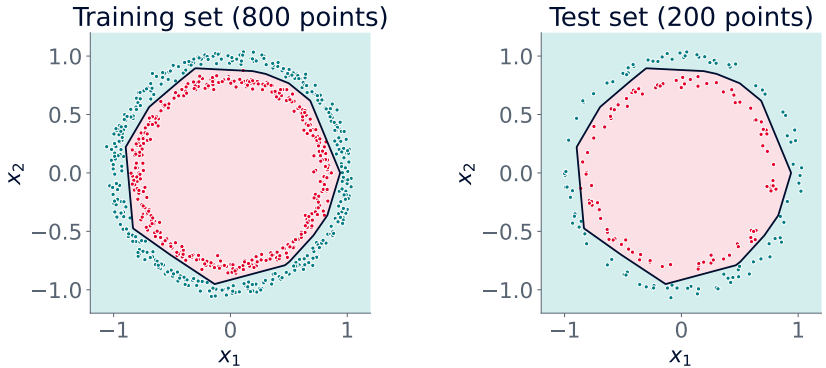
```
model_1.eval()                              # 1
with torch.inference_mode():                # 2
    logits = model_1(X_test).squeeze(-1)
    test_loss = loss_fn(logits, y_test)  # 3

    pred = (logits >= 0).long()             # 4
    correct = (pred == y_test).float()
    accuracy = correct.mean()
```

No `backward()` or `step()` here. For percent accuracy, multiply by 100.

1. **Evaluation mode.** Disable dropout. `eval()` alone still allows gradients.
2. **Skip gradient recording.** Compute predictions without building the computation graph needed by `backward()`.
3. **Measure performance.** Compute logits and loss with fitted weights fixed.
4. **Predict labels.** Logit ≥ 0 means probability ≥ 0.5. Accuracy is the fraction correct.

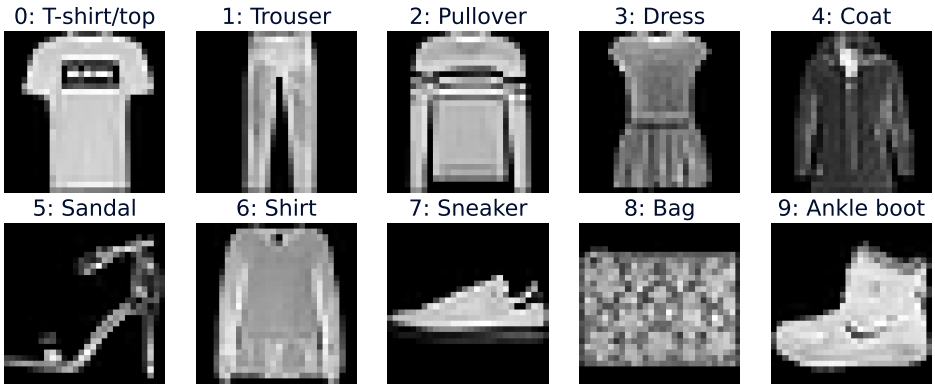
The learned circle boundary



Background: predicted class. Points: observed class. Line: $P(y = 1 \mid x) = 0.5$.

Recorded test result: 100.0% accuracy; binary cross-entropy 0.0281.

FashionMNIST: ten classes



Each observation is a 28×28 grayscale image.

The MLP receives 784 pixel values and outputs ten class scores.

FashionMNIST inputs

```
from torchvision import datasets
from torchvision.transforms import ToTensor

train_data = datasets.FashionMNIST(
    "data", train=True, download=True, transform=ToTensor())
test_data = datasets.FashionMNIST(
    "data", train=False, download=True, transform=ToTensor())

image, label = train_data[0]
image.shape  # (1, 28, 28): channel, height, width
```

- `ToTensor()` converts these byte-valued images to float32 values in $[0, 1]$.
- Labels are integer class indices from 0 to 9.
- The official split provides 60,000 development images and 10,000 test images.

Datasets and data splits

```python
from torch.utils.data import (
    random_split, DataLoader)

# 1. Retrieve one example
image, label = train_data[0]
# image: (1, 28, 28); label: integer 0-9

# 2. Fix split membership
split_rng = torch.Generator().manual_seed(42)
train_set, val_set = random_split(
    train_data, [54000, 6000],
    generator=split_rng)

# Train / validation / test: 54000 / 6000 / 10000
```

`random_split` returns subsets. It does not stratify by class.

1. **Dataset: one example.** Indexing retrieves an image and its label, applying `ToTensor()`.
2. **Separate the roles.** Training fits weights. Validation selects a checkpoint. Test data gives the final assessment.
3. **Fix the split once.** The seeded generator reproduces membership. Loader shuffling later changes only the order.

Building DataLoaders

```
train_loader = DataLoader(
    train_set,
    batch_size=64,          # 1
    shuffle=True,           # 2
    num_workers=0,          # 3
    drop_last=False,        # 4
    generator=torch.Generator().manual_seed(42))

val_loader = DataLoader(
    val_set, batch_size=64, shuffle=False)
test_loader = DataLoader(
    test_data, batch_size=64, shuffle=False)
```

Move batches to `device` inside the training or evaluation loop.

Official API: `torch.utils.data.DataLoader`.

1. **Batch size.** Retrieve and stack up to 64 image–label pairs per iteration.
2. **Shuffle training data.** Use a new random order each pass. Image–label pairs remain together.
3. **Loading workers.** Zero loads data in the main process. Positive values use worker processes.
4. **Keep the remainder.** `drop_last=False` retains the smaller final batch. It is the default.

A batch, a step, an epoch

```
# 1. Begin a pass; fetch one batch
batch_iterator = iter(train_loader)
images, labels = next(batch_iterator)

# 2. A new leading axis indexes examples
images.shape  # (64, 1, 28, 28)
labels.shape  # (64,)

# 3. Images and batches are different counts
len(train_loader.dataset)  # 54,000 images
len(train_loader)          # 844 batches
```

$54{,}000 = 843 \times 64 + 48$: 844 batches per epoch, with 48 images in the final batch.

1. **Iteration.** `iter()` starts a pass. `next()` fetches the next batch. A `for` loop repeats this automatically.
2. **Batch.** Images gain a batch axis. Label i belongs to image i. Fetching a batch does not train the model.
3. **Step and epoch.** One `optimizer.step()` is one update. One full pass through this loader is one epoch.

The FashionMNIST MLP

```
class FashionMNIST_MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),  # (B, 1, 28, 28) -> (B, 784)
            nn.Linear(784, 256), nn.ReLU(),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Linear(64, 10))  # Ten raw logits
    def forward(self, x):
        return self.net(x)
model_2 = FashionMNIST_MLP().to(device)
```

`Flatten` preserves the batch axis. `Sequential` registers all layers.

The final layer has no softmax: `CrossEntropyLoss` handles it internally.

Multiclass logits and loss

```
logits = model_2(images)              # (B, 10)
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(logits, labels)        # labels: int64, (B,)

probability = logits.softmax(dim=1)  # Sum to 1 across classes
prediction = logits.argmax(dim=1)    # Shape (B,)
```

$$p_{ik} = \frac{\exp(a_{ik})}{\sum_j \exp(a_{ij})}, \qquad \ell_i = -\log p_{i,y_i}.$$

- Integer label 3 selects the fourth class. One-hot labels are unnecessary here.
- `dim=1` is the class axis in a (B, K) score matrix.
- The loss takes raw scores. Softmax preserves their order, so argmax can use logits.

Training with mini-batches

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(
    model_2.parameters(), lr=0.02)

for epoch in range(10):
    model_2.train()                    # 1
    for images, labels in train_loader:  # 2
        images = images.to(device)    # 3
        labels = labels.to(device)
        logits = model_2(images)      # 4
        loss = loss_fn(logits, labels)
        optimizer.zero_grad()         # 5
        loss.backward()
        optimizer.step()
```

Ten epochs give 8440 updates. Validate after each epoch.

1. **Start an epoch.** Restore training behavior after validation.
2. **Loop over batches.** Each iteration retrieves a batch of `images` and their corresponding `labels`.
3. **Match devices.** Move the batch to the device holding the model.
4. **Forward and loss.** Produce ten logits per image. Average cross-entropy over this batch.
5. **Fit this batch.** Clear gradients, backpropagate the loss, and update parameters once.

Evaluation across batches

```
# still within the for loop
model_2.eval()                          # 1
loss_sum, correct, n = 0.0, 0, 0        # 2
with torch.inference_mode():            # 1
    for images, labels in val_loader:
        images = images.to(device)
        labels = labels.to(device)
        logits = model_2(images)
        batch_loss = loss_fn(logits, labels)
        B = labels.size(0)
        loss_sum += batch_loss.item() * B  # 3
        pred = logits.argmax(dim=1)
        correct += (pred == labels).sum().item()
        n += B                          # 4
val_loss = loss_sum / n                 # 5
val_accuracy = 100 * correct / n
```

Final test: use `test_loader`.

1. **Evaluation settings.** Select evaluation behavior and skip gradient recording.
2. **Reset totals.** Start fresh for this pass.
3. **Sum image losses.** Multiply the batch mean by B. `item()` extracts a number.
4. **Count outcomes.** Argmax selects the class. Count correct predictions and all images.
5. **Average once.** Divide by n to give every image equal weight.

Selecting a checkpoint

```python
from copy import deepcopy

# 1. Before the training loop
best_loss = float("inf")
best_state = None

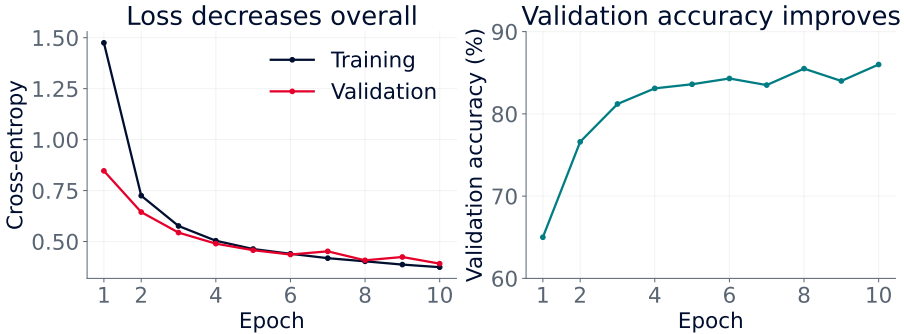
# 2. After each epoch's validation
if val_loss < best_loss:
    best_loss = val_loss
    best_state = deepcopy(model_2.state_dict())

# 3. After all epochs
model_2.load_state_dict(best_state)
```

This selects a checkpoint. Early stopping would also end training.

1. **Initialize once.** Starting at infinity lets the first finite validation loss set the best value.
2. **Keep the best snapshot.** A lower validation loss replaces it. `deepcopy` preserves weights while training continues.
3. **Restore, then test.** Load the chosen weights and evaluate them on test data once.

Learning curves

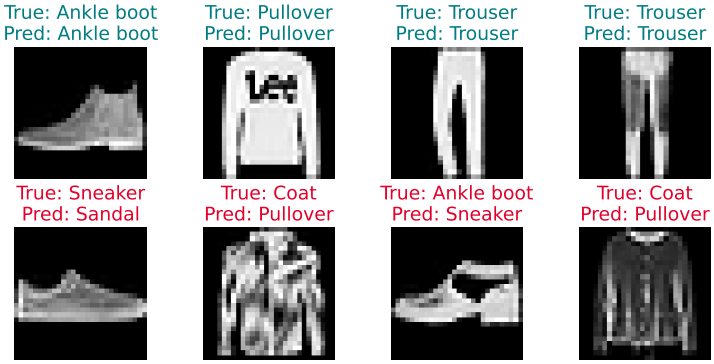


Selected checkpoint: epoch 10, the smallest validation loss in this run.

Validation loss fluctuates; one upward step alone is not a persistent overfitting trend.

Recorded notebook run: batch size 64, SGD learning rate 0.02. Training loss averages pre-update batches; validation uses the end-of-epoch model.

Inspect the predictions



Full test set: 85.01% accuracy; cross-entropy 0.4152.

Top: four correct predictions. Bottom: four errors, selected to inspect confusions.

Dropout in code

```
hidden = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(p=0.2)
)
```

$$\widetilde{h} = \frac{r \odot h}{1-p}, \qquad r_j \sim \text{Bernoulli}(1-p).$$

- In training, each forward pass samples masks and rescales retained entries.
- In evaluation, dropout returns its input unchanged.
- Autograd uses the sampled mask in the backward pass automatically.

After class: try to add dropout in `FashionMNIST_MLP`. Compare with the current model.

Weight initialization

Class-based circle model: optional initialization before fitting.

```
model_1 = TwoclassClassificationMLP().to(device)

for layer in [model_1.hidden1, model_1.hidden2]:
    nn.init.kaiming_normal_(layer.weight, nonlinearity="relu")
    nn.init.zeros_(layer.bias)
# Final affine output layer:
nn.init.xavier_normal_(model_1.output.weight)
nn.init.zeros_(model_1.output.bias)
```

$$\text{He: } \operatorname{Var}(W_{jk}) = \frac{2}{d_{\text{in}}}, \qquad \text{Xavier (gain 1): } \operatorname{Var}(W_{jk}) = \frac{2}{d_{\text{in}} + d_{\text{out}}}.$$

- The trailing underscore means the initializer changes the tensor in place.
- Random weights break symmetry. Zero biases are usually acceptable.

Saving and restoring weights

```
torch.save(model_2.state_dict(), "fashion_mlp.pth")
loaded = FashionMNIST_MLP()  # Recreate the same architecture
state = torch.load("fashion_mlp.pth", map_location="cpu",
                   weights_only=True)
loaded.load_state_dict(state)
loaded.to(device).eval()
with torch.inference_mode():
    logits = loaded(images.to(device))
```

- `state_dict()` stores parameter and persistent-buffer values.
- Layer names must match, including a `Sequential` container's names.
- `map_location` allows loading without the original GPU.

To resume training with optimizer history, save `optimizer.state_dict()` and the epoch too.

Common code checks

Symptom / risk	What to check
Wrong loss shape	Binary: logits and targets $(B,)$. Multiclass: logits (B, K) and targets $(B,)$.
Unexpected broadcasting	For circles, compare labels and predictions both shaped $(B,)$; avoid $(B, 1)$ versus $(B,)$.
Loss has no gradient	Keep the loss connected to parameters. Log a separate `loss.item()`.
Model does not update	Optimizer references the current model. Run zero_grad, backward, then step.
Device or dtype error	Match model/data devices. Binary targets: float. Class indices: int64.
Unstable or slow learning	Check data scale. Compare learning rates on validation data.